

UV: The Next-Gen Python Package Manager

✓ What is `uv`?

`uv` is a **blazing-fast Python package manager**, built as a modern alternative to tools like `pip`, `pipenv`, and `poetry`.

Created by Astral (the team behind Ruff), `uv` focuses on:

-  **Extreme speed** (written in Rust)
-  Advanced dependency resolution (like `poetry`)
-  Direct package installation
-  Reproducible lockfile generation

Installing `uv` with Homebrew (macOS / Linux)

```
bash
```

 Copiar  Editar

```
brew install astral-sh/uv/uv
```

 If you don't have `brew`, you can install it from: <https://brew.sh>

Basic `uv` Usage

1. Install a package and add it to `pyproject.toml`

bash

 Copiar  Editor

```
uv pip install pandas
```

This acts like a combination of `pip install` and `poetry add`.

2. Install from a `requirements.txt` file

bash

 Copiar  Editor

```
uv pip install -r requirements.txt
```

Great for traditional projects not using `pyproject.toml`.

3. Generate a locked requirements file

bash

 Copiar  Editar

```
uv pip compile pyproject.toml
```

Creates a pinned `requirements.txt` (like `pip-tools`).

4. Create a virtual environment

bash

 Copiar  Editar

```
uv venv
```

Creates a clean virtual environment in `.venv`.

5. Other Features

- Full `pip` replacement:

```
bash
```

 Copiar  Editar

```
uv pip install numpy
```

- Supports `-e` editable installs
- Works without wheels when needed
- Offline installs if packages are cached

Why Use `uv`?

Feature	<code>uv</code>	Traditional <code>pip</code> / <code>venv</code>
Speed	 Extremely fast	 Slower
Dependency resolution	 Advanced (like <code>poetry</code>)	 Basic
Reproducible locking	 Yes	 No
Virtual environments	 Built-in (<code>uv venv</code>)	 Separate (<code>python -m venv</code>)
Modern project support	 Uses <code>pyproject</code>  1	 Mostly <code>requirements.txt</code>